

Understanding What is HTTP?

9 min read · Sep 23, 2023



WebDevSumoz

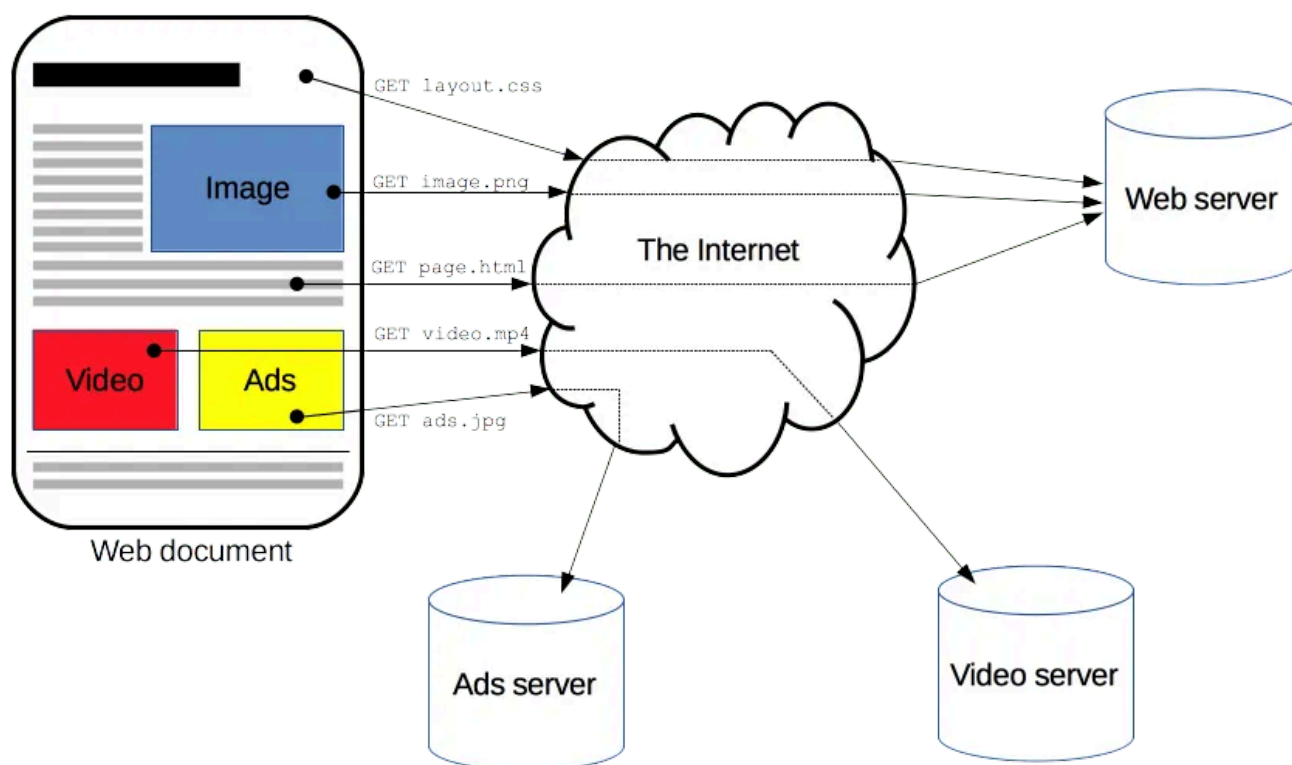
Follow



Listen



Share

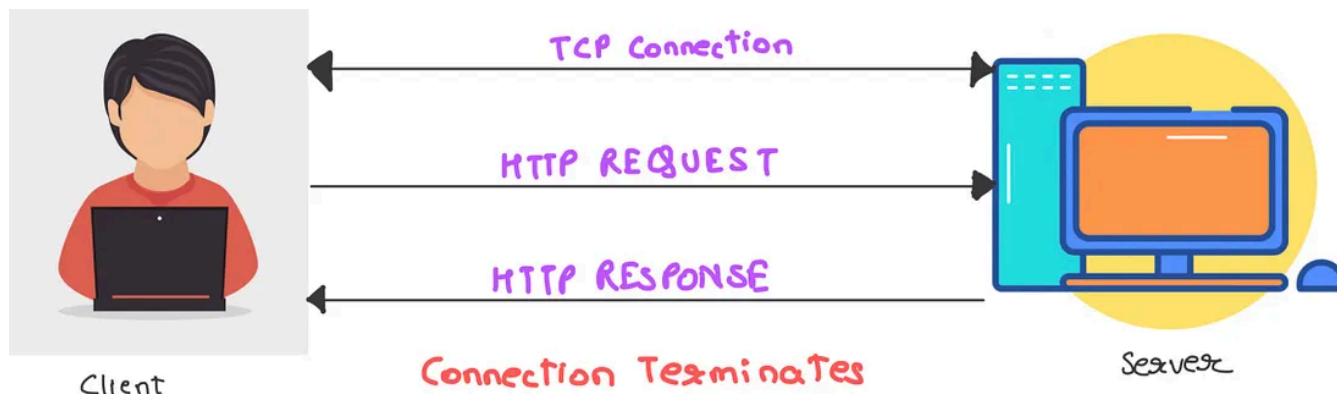


HTTP is a client-server protocol for fetching resources such as HTML documents. It is the foundation of any data exchange on the Web and it is a client-server protocol, which means requests are initiated by the recipient, usually the Web browser.

A protocol is a system of rules that define how data is exchanged within or between computers. Communications between devices require that the devices agree on the format of the data that is being exchanged. The set of rules that defines a format is called a protocol.

In client-server protocols, like HTTP (HTTP/1.0), sessions consist of Four phases:

1. The client establishes a TCP connection (or the appropriate connection if the transport layer is not TCP).
2. The client sends its request, and waits for the answer.
3. The server processes the request, sending back its answer, providing a status code and appropriate data.
4. Connection gets terminated.



Before diving into HTTP further, we will take a look *What is TCP and how a TCP connection is established between client and server?*

Well, TCP (Transmission Control Protocol) that lets two hosts connect and exchange data streams. TCP guarantees the delivery of data and packets in the same order as they were sent.

How a TCP connection established between client and server?

TCP (Transmission Control Protocol) uses a three-way handshake to set up a TCP/IP connection over an IP based network.

The three messages transmitted by TCP to negotiate and start a TCP session are nicknamed SYN, SYN-ACK, and ACK for SYNchronize, SYNchronize-ACKnowledgement, and ACKnowledge respectively. The three message mechanism is designed so that two computers that want to pass information back and forth to each other can negotiate the parameters of the connection before transmitting data such as HTTP browser requests.

TCP Three-way handshake:

1. The host, generally the browser, sends a TCP SYNchronize packet to the server.

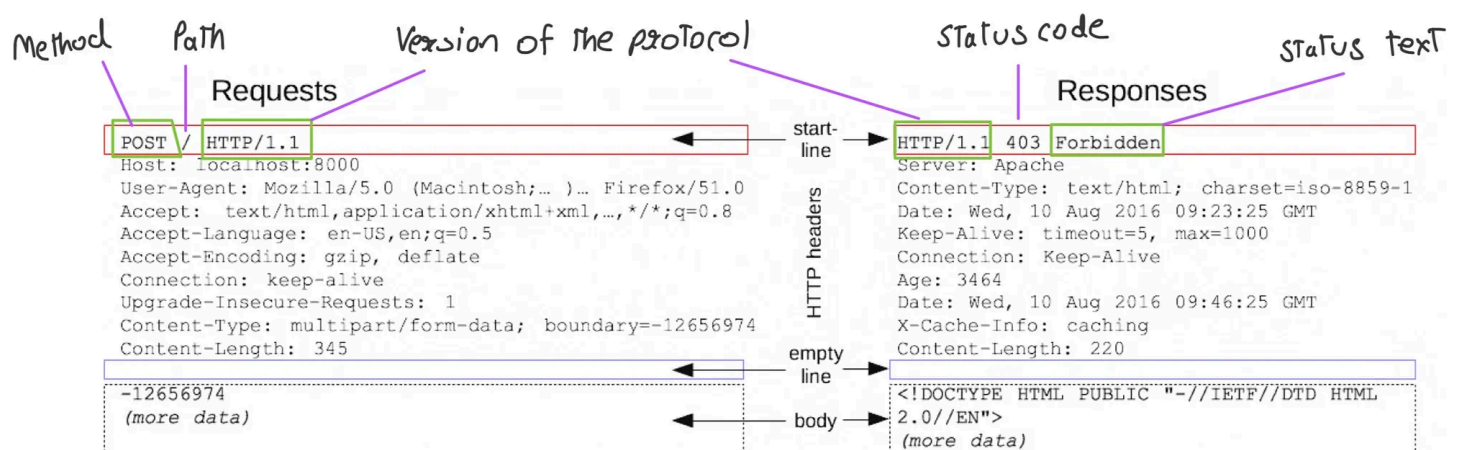
2. The server receives the SYN and sends back a SYNchronize-ACKnowledgement.
3. The host receives the server's SYN-ACK and sends an ACKnowledge. The server receives ACK and the TCP socket connection is established.



Now, that we know how TCP connection is established. Let's look at how client sends the request to the server. well for client and server to communicate they use HTTP messages.

Http message

HTTP messages are how data is exchanged between a server and a client. There are two types of messages: *requests* sent by the client to trigger an action on the server, and *responses*, the answer from the server.



HTTP requests, and responses, share similar structure and are composed of 4 parts:

1. A *start-line* describing the requests to be implemented, or its status of whether successful or a failure. This start-line is always a single line.

2. An optional set of *HTTP headers* specifying the request, or describing the body included in the message.
3. A blank line indicating all meta-information for the request has been sent.
4. An optional *body* containing data associated with the request (like content of an HTML form), or the document associated with a response. The presence of the body and its size is specified by the start-line and HTTP headers.

The start-line and HTTP headers of the HTTP message are collectively known as the *head* of the requests, whereas its payload is known as the *body*.

If you want to know more about internal structure of HTTP message then continue reading or else you can skip to next section.

HTTP Request Message

Start Line

HTTP requests are messages sent by the client to initiate an action on the server. Their *start-line* contain three elements:

1. **HTTP Method:** a verb like GET, PUT or POST, that describes the action to be performed. For example, GET indicates that a resource should be fetched or POST means that data is pushed to the server (creating or modifying a resource, or generating a temporary document to send back).
2. The Path of the protocol, port, and domain are usually characterized by the request context. The format of this Path varies between different HTTP methods.

An **absolute path**, ultimately followed by a '?' and query string. This is the most common form, known as the *origin form*, and is used with GET, POST, HEAD, and OPTIONS methods.

- POST / HTTP/1.1
- GET /background.png HTTP/1.0
- HEAD /test.html?query=alibaba HTTP/1.1
- OPTIONS /anypage.html HTTP/1.0

- A **complete URL**, known as the *absolute form*, is mostly used with `GET` when connected to a proxy. `GET https://developer.mozilla.org/en-US/docs/Web/HTTP/Messages HTTP/1.1`

The **authority component** of a URL, consisting of the domain name and optionally the port (prefixed by a `:`), is called the *authority form*. It is only used with `CONNECT` when setting up an HTTP tunnel. `CONNECT developer.mozilla.org:80 HTTP/1.1`

The *asterisk form*, a simple asterisk (`*`) is used with `OPTIONS`, representing the server as a whole. `OPTIONS * HTTP/1.1`

Get WebDevSumoz's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

3. The *HTTP version*, which defines the structure of the remaining message, acting as an indicator of the expected version to use for the response. Examples of HTTP versions are HTTP/1.0, HTTP/1.1, HTTP/2, and HTTP/3.

HTTP Header

```
POST / HTTP/1.1
Host: localhost:8000
User-Agent: Mozilla/5.0 (Macintosh;... )... Firefox/51.0
Accept: text/html,application/xhtml+xml,...,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Content-Type: multipart/form-data; boundary=-12656974
Content-Length: 345
```

Request headers

General headers

Representation headers

-12656974
(more data)

Fig. HTTP Header for Request message.

Request Header: A request header is an HTTP header that can be used in an HTTP request to provide information about the request context, so that the server can tailor the response. For example, the Accept-* headers indicate the allowed and

preferred formats of the response. Other headers can be used to supply authentication credentials (e.g. Authorization), to control caching, or to get information about the user agent or referrer, etc.

General Header: Its is an outdated term used to refer to an HTTP header which is used in both request and response message header. Now, general headers are simply referred to as response or request headers.

Representation Headers: like Content-Type that describe the original format of the message data and any encoding applied.

Body

The final part of the request is its body. Not all requests have one: requests fetching resources, like `GET`, `HEAD`, `DELETE`, or `OPTIONS`, usually don't need one. Some requests send data to the server in order to update it: as often the case with `POST` requests (containing HTML form data)

HTTP Response Message

Start line (Status line)

The start line of an HTTP response, called the *status line*, contains the following information:

1. The *protocol version*, usually `HTTP/1.1`.
2. A *status code*, indicating success or failure of the request. Common status codes are `200`, `404`, or `302`
3. A *status text*. A brief, purely informational, textual description of the status code to help a human understand the HTTP message.

A typical status line looks like: `HTTP/1.1 404 Not Found`.

HTTP Header

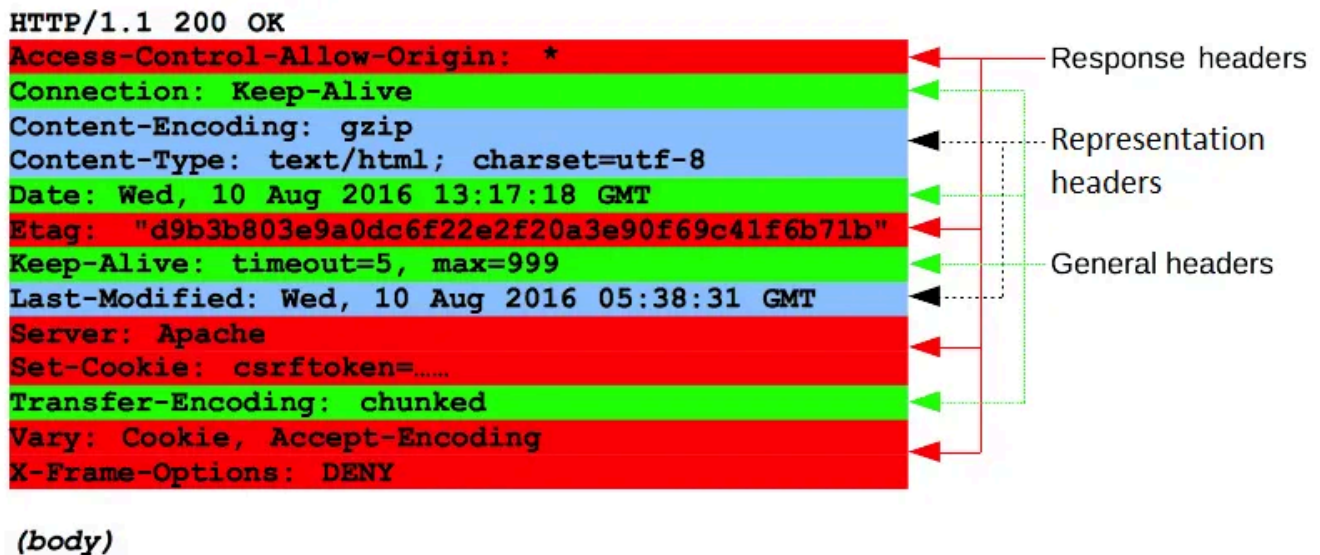


Fig. HTTP Header for Response message

Response Header: A response header is an HTTP header that can be used in an HTTP response and that doesn't relate to the content of the message. Response headers, like Age, Location or Server are used to give a more detailed context of the response.

Representation Header: A representation header describes to the client the particular *representation* of the resource sent by the server in an HTTP message body.

Body: The last part of a response is the body. Not all responses have one: responses with a status code that sufficiently answers the request without the need for corresponding payload (like 201 Created Or 204 No Content) usually don't.

How Connection Management is done HTTP/1.x?

Opening and maintaining connections largely impacts the performance of websites and Web applications. In HTTP/1.x, there are several models: *short-lived connections*, *persistent connections*, and *HTTP pipelining*.

HTTP mostly relies on TCP for its transport protocol, providing a connection between the client and the server. In its infancy i.e versions less than equal to HTTP/1.0, HTTP used a single model to handle such connections. These connections were short-lived: a new one created each time a request needed sending, and closed once the answer had been received.

Opening each TCP connection is a resource-consuming operation. Several messages must be exchanged between the client and the server. Network latency and

bandwidth affect performance when a request needs sending. Modern Web pages require many requests (a dozen or more) to serve the amount of information needed, proving this earlier model inefficient.

Two newer models were created in *HTTP/1.1*

As of HTTP/1.1, the connection is no longer closed after completing the third phase, and the client is now granted a further request: this means the second and third phases can now be performed any number of times.

The *persistent-connection* model keeps connections opened between successive requests, reducing the time needed to open new connections.

The *HTTP pipelining model* goes one step further, by sending several successive requests without even waiting for an answer, reducing much of the latency in the network

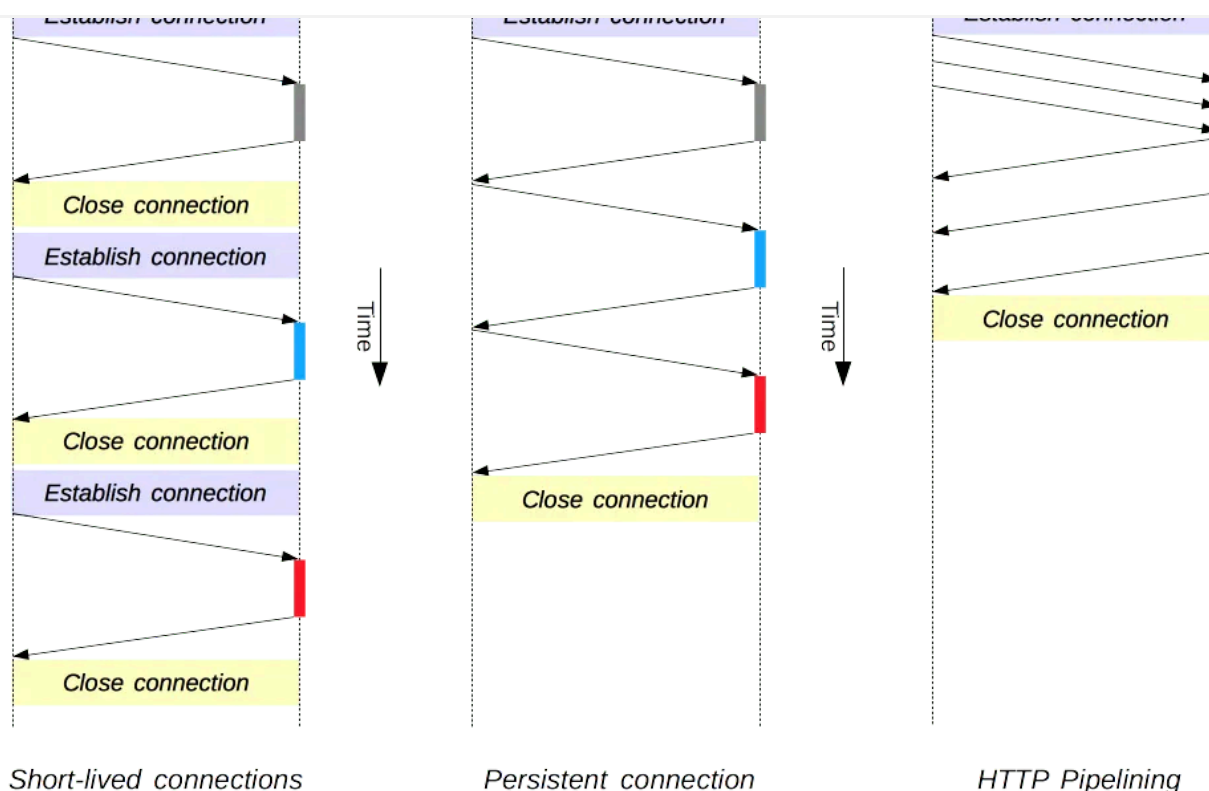
Open in app ↗

Sign up

Sign in

Medium

Search



Short-lived connections:

The original model of HTTP, and the default one in HTTP/1.0, is *short-lived connections*. Each HTTP request is completed on its own connection; this means a TCP handshake happens before each HTTP request, and these are serialized.

The TCP handshake itself is time-consuming, but a TCP connection adapts to its load, becoming more efficient with more sustained (or warm) connections. Short-lived connections do not make use of this efficiency feature of TCP, and performance degrades from optimum by persisting to transmit over a new, cold connection.

Persistent connections:

Short-lived connections have two major hitches: the time taken to establish a new connection is significant, and performance of the underlying TCP connection gets better only when this connection has been in use for some time (warm connection). To ease these problems, the concept of a *persistent connection* has been designed, even prior to HTTP/1.1. And, HTTP/1.1 use it as its default implementation. Alternatively this may be called a *keep-alive connection*.

A persistent connection is one which remains open for a period of time, and can be reused for several requests, saving the need for a new TCP handshake, and utilizing TCP's performance enhancing capabilities. This connection will not stay open forever: idle connections are closed after some time (a server may use the Keep-Alive header to specify a minimum time the connection should be kept open).

Persistent connections also have drawbacks; even when idling they consume server resources, and under heavy load, DoS attacks can be conducted. In such cases, using non-persistent connections, which are closed as soon as they are idle, can provide better performance.

HTTP pipelining:

By default, HTTP requests are issued sequentially. The next request is only issued once the response to the current request has been received. As they are affected by network latencies and bandwidth limitations, this can result in significant delay before the next request is *seen* by the server. This is called as *Head of Line Blocking* (HOL) problem.

Pipelining is the process to send successive requests, over the same persistent connection, without waiting for the answer. This avoids latency of the connection.

Not all types of HTTP requests can be pipelined: only idempotent methods, that is GET , HEAD , PUT and DELETE , can be replayed safely. Should a failure happen, the pipeline content can be repeated.

Note: HTTP pipelining is not activated by default in modern browsers:

Buggy proxies are still common and these lead to strange and erratic behaviors that Web developers cannot foresee and diagnose easily.

Pipelining is complex to implement correctly: the size of the resource being transferred, the effective RTT that will be used, as well as the effective bandwidth, have a direct incidence on the improvement provided by the pipeline. Without knowing these, important messages may be delayed behind unimportant ones. The notion of important even evolves during page layout! HTTP pipelining therefore brings a marginal improvement in most cases only.

I would recommend reading the following article about HTTP/1.0, HTTP/2, and HTTP/3.

HTTP/1 to HTTP/2 to HTTP/3

It's not about being the best, it's about being better than you were yesterday.

medium.com

References:

HTTP/1 to HTTP/2 to HTTP/3

It's not about being the best, it's about being better than you were yesterday.

medium.com

Http Client

Http

Http Request

Http Response

Tcp

[Follow](#)

Written by WebDevSumoz

152 followers · 114 following

A Front-End Engineer with professional experience of 7 years. I'm here to share my learnings and experience in tech industry.

No responses yet



Write a response

What are your thoughts?

More from WebDevSumoz





In Stackademic by WebDevSumoz

Walmart Karat Interview | Software Engineer- UI 3

In October 2024, I had the opportunity to interview with Walmart for the Software Engineer—UI 3 role. The first round was a Karat...



Oct 17, 2024



50



2



WebDevSumoz

How To Test React Query Hook using Jest

As I make my way through the Jest testing for the React Hooks, I want to share my experience with you all. Please let me know If I can do...

Jul 25, 2024



8



Bloomberg



WebDevSumoz

Bloomberg Senior Front-End Engineer — Phone Interview

I applied to Bloomberg's Senior Front-End Engineer role via LinkedIn, without requesting a referral. Shortly after, I received an email to...



Jul 31



8



WebDevSumoz

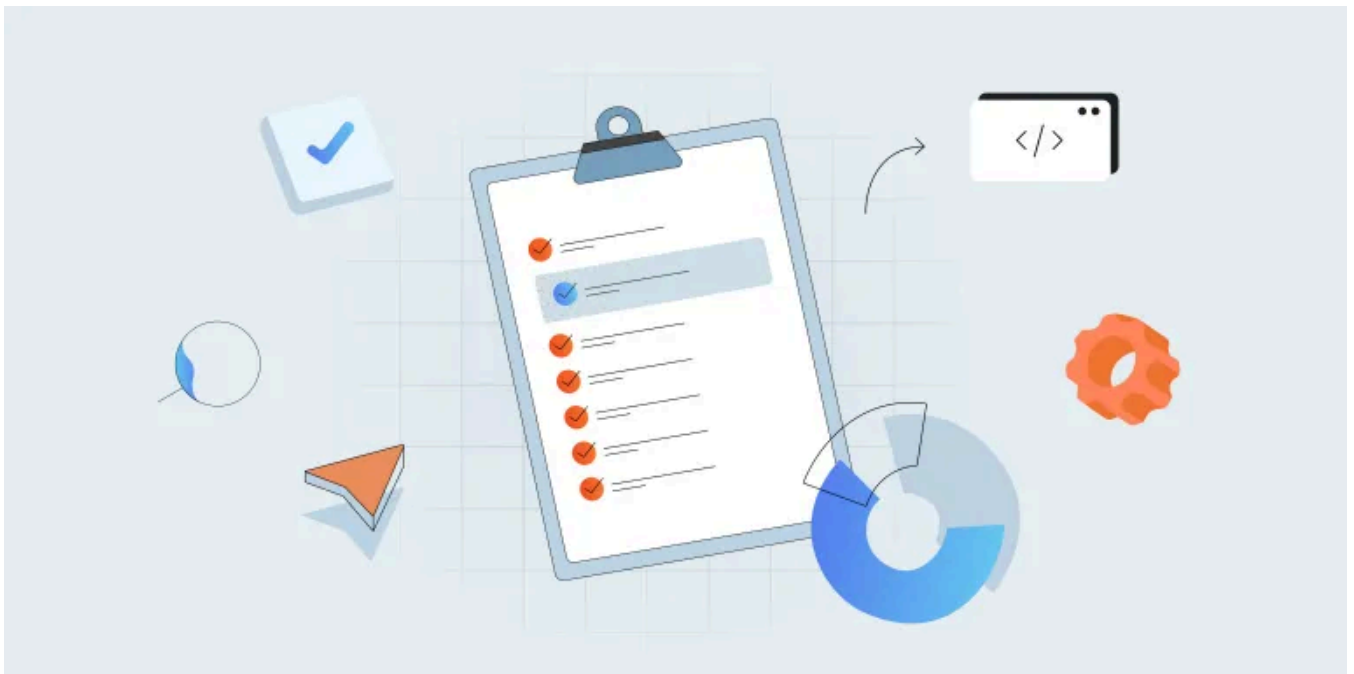
Most Asked Async JavaScript Interview Questions

These are top Promise/Async questions that I practiced for my Front End interview preparation. I would highly recommend first reading about...

★ Dec 30, 2024 🖱 7 💬 2

[See all from WebDevSumoz](#)

Recommended from Medium

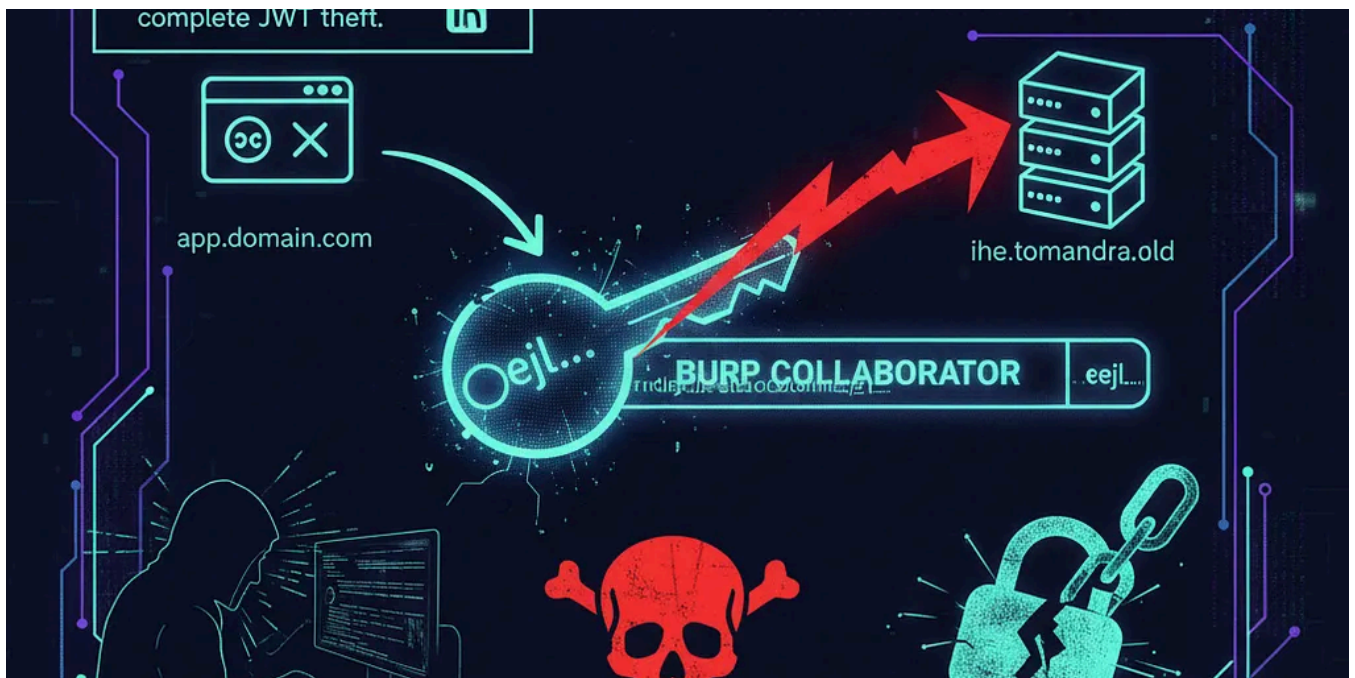
 Sandra Parker

How and Why to Perform Configuration Testing

Imagine getting geared up to launch a beautifully designed website or application for months only to realize that your application works...

★ 4d ago





 In InfoSec Write-ups by Shah kaif

Stealing JWT Tokens via OAuth redirect_uri Manipulation: A Critical Vulnerability

By Shah kaif | "A single redirect_uri = complete JWT theft." | LinkedIn

3d ago  11  1



 In ILLUMINATION by om kumar

How I Found Critical Flaws in a Medical SOAP API

My €400 Bug Bounty Story

★ May 6 🖱 7

 JigNect | Software Testing Company

Web Application Security Testing: The Ultimate Guide to Protecting Web Apps

Think of your web app as the front door to your digital business. Your app is always “live,” processing logins, transactions, and private...

Jun 25 🖱 6



Insufficient JSON Payload Validation: Real-World API Exploits



API

```
{ username: john  
  password:  
    Password123  
  } role: admin
```

 Israel Aráoz Severiche

Hacking APIs: Insufficient JSON Payload Validation

APIs are the backbone of modern applications. From mobile apps to SaaS platforms, APIs connect services and users at scale. But many REST...

★ Sep 30 🖱 7



In MeetCyber by Ayush

How I found SQL Injection on State Government

I found sql injection on Indian state govt which exposed whole ministry passwords, username, access token...

★ 4d ago 🖱 105



See more recommendations